



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 2 – Overloading and Inheritance
Topic – 2.3 Function Overloading

Prof. V.N.Nirgude
Assistant Professor

E-mail :

nirgudevikascomp@sanjivani.org.in

Contact No: 9975240215



Function Overloading

- If any class have multiple functions with same names but different parameters then they are said to be overloaded.
- Function overloading allows you to use the same name for different functions, to perform, either same or different functions in the same class.
- If you have to perform one single operation but with different number or types of arguments, then you can simply overload the function.
- When an overloaded function is called, the C++ compiler selects the proper function by examining the number, types and order of arguments in the function call.



Function Overloading with Number of Arguments are different

- In this, we can define two functions with same names but different number of parameters of the same data type.

- Example

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
int sum(int x, int y, int z)
{
    cout << x+y+z;
}
```

Here sum() function is overloaded, to have two and three arguments. Which sum() function will be called, depends on the number of arguments.



Function Overloading with Different Data type of Arguments

- In this type of overloading we define two or more functions with same name and same number of parameters, but the type of parameter is different.

```
int sum(int x,int y)
{
    cout<< x+y;
}
double sum(double x, double y)
{
    cout << x+y;
}
```

```
int main( )
{
    demo d;
    d.sum(10,20);
    d.sum(10.5,20.5);
}
```



Program to overload square function which calculate square of an integer and square of double

```
#include<iostream>
using namespace std;
int square(int x)
{
    cout<<"Square of integer"<<x<<"is";
    return x*x;
}
double square(double y)
{
    cout<<"Square of double"<<y<<"is";
    return y*y;
}
int main()
```

```
{
    int a;
    double b;
    a = square(7);
    cout<< a;
    b = square(7.5);
    cout<< b;
    return 0;
}

Output:
Square of integer 7 is 49
Square of double 7.5 is 56.25
```



- Two functions cannot be overloaded when the only difference is that one takes a reference parameter and the other takes a normal, call-by-value parameter.

- Example:

```
void fun(int x);
```

```
void fun(int &x); // error
```



Rules for using Default Arguments

- If you pass default argument, then you will have to pass default values for all the subsequent arguments (from right to left).
 - `sum (int x,int y=0); //correct`
 - `sum (int x,int y=0,int z); // This is incorrect`
 - `sum (int x,int y=10,int z=10); // Correct`

Thank You..